

Imagized Language Modeling with Product Codes, QA Alignment, and Diffusion Denoising

A Deep Technical Analysis

Incoder Research Log

October 21, 2025

Abstract

We document and analyse the current implementation of the Imagized Language Model (ILM) stack in this repository. The system learns visual glyph embeddings, differentiable product “colour codes”, question–answer (QA) alignment, and a 2D diffusion-based infiller. We derive the training objectives, explain why optimisation nudges the model toward semantic organisation, and outline how the same pipeline can be applied to under-documented scripts such as Oracle Bone, Tangut (Xixia), or Khitan (Qidan) without dictionaries.

1 System Overview

The ILM pipeline is modular :

1. **Glyph Encoding:** Convert every token (word or character) into a 128×128 RGB glyph image combining ASCII row–column coding and centred font rendering (`ilm/utils/glyphs.py`).
2. **Product Code Learning:** Train a 3×32 differentiable product code layer that maps glyph CNN features to discrete “colour” indices (`ilm/code/product.py`, `scripts/train_color_codes.py`).
3. **QA Alignment:** Fine-tune colour codes with InfoNCE on question–answer pairs plus Gram matrix autocorrelation, enforcing structural alignment of semantics (`scripts/train_codes_from_qa.py`).
4. **Sentence Diffusion:** Assemble glyph embeddings into $H \times W$ frames and train a small UNet to reconstruct masked features, enabling denoising-based language generation (`scripts/train_diffusion.py`).

Pipeline summary. Figure placeholder omitted; see main documentation for visuals.

2 Differentiable Product Codes

2.1 Glyph Encoder

A lightweight CNN, $f_{\text{glyph}} : \mathbb{R}^{3 \times 128 \times 128} \rightarrow \mathbb{R}^{d_g}$, is defined (`ilm/encoders/glyph_cnn.py`) with $d_g = 128$. Convolutional layers reduce spatial resolution while preserving writing strokes.

2.2 Product Code Layer

The product code module maintains $C = 3$ codebooks $E_c \in \mathbb{R}^{K \times d_e}$ with $K = 32$ and $d_e = 128$. For each channel c , logits $\ell_c = W_c g + b_c$ are produced, where $g = f_{\text{glyph}}(x)$.

Sampling uses relaxed Gumbel–Softmax:

$$y_c = \text{GumbelSoftmax}(\ell_c, \tau), \quad e = \sum_{c=1}^C y_c E_c. \quad (1)$$

With straight-through estimation the forward pass is effectively discrete while gradients flow through the soft relaxation.

2.3 Losses

InfoNCE (Glyph $\leftrightarrow$ Code). For a batch of B glyphs with features g_i and code embeddings e_i , we use

$$\mathcal{L}_{\text{NCE}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\langle \hat{g}_i, \hat{e}_i \rangle / \tau_{\text{info}})}{\sum_{j=1}^B \exp(\langle \hat{g}_i, \hat{e}_j \rangle / \tau_{\text{info}})}, \quad (2)$$

where hats denote ℓ_2 normalization and $\tau_{\text{info}} = 0.07$. This pushes each code embedding toward its glyph feature and away from others, forcing codes that preserve glyph identity.

Usage KL. To discourage collapse, average channel assignments $p_c = \frac{1}{B} \sum_i y_{c,i}$ are matched to the uniform distribution $u = \frac{1}{K} \mathbf{1}$:

$$\mathcal{L}_{\text{usage}} = \frac{1}{C} \sum_{c=1}^C \text{KL}(p_c \| u). \quad (3)$$

This keeps each axis of the product space populated, ensuring the memory table has high capacity.

Independence Penalty. Cross-covariance between channel assignments is penalised:

$$\mathcal{L}_{\text{indep}} = \frac{2}{C(C-1)} \sum_{a < b} \|\text{Cov}(y_a, y_b)\|_F^2. \quad (4)$$

This encourages factorised structure so that each channel captures different semantics (grammar, meaning, tone, etc.).

Total Objective. Training minimises

$$\mathcal{L}_{\text{codes}} = \mathcal{L}_{\text{NCE}} + \lambda_{\text{usage}} \mathcal{L}_{\text{usage}} + \lambda_{\text{indep}} \mathcal{L}_{\text{indep}}, \quad (5)$$

with $(\lambda_{\text{usage}}, \lambda_{\text{indep}}) = (0.1, 0.1)$ in `configs/color.yaml`. The gradient encourages glyphs with similar strokes or structures to share nearby indices, while maintaining combinatorial expressivity.

3 QA Alignment: Why Semantics Emerge

3.1 Batch Collation

For each QA sample all tokens are deduplicated, the glyph DB provides or creates glyph PNGs, and stacked into a mini-batch X .

3.2 Loss Components

Let E denote token code embeddings and G glyph features. For each question Q and answer A we compute pooled representations

$$e_Q = \frac{1}{|Q|} \sum_{i \in Q} E_i, \quad e_A = \frac{1}{|A|} \sum_{j \in A} E_j, \quad (6)$$

(normalised before similarity).

QA InfoNCE. Pairs are treated as positives, negatives come from the rest of the mini-batch:

$$\mathcal{L}_{QA} = -\frac{1}{B} \sum_{b=1}^B \log \frac{\exp(\langle \hat{e}_b^Q, \hat{e}_b^A \rangle / \tau_{QA})}{\sum_{j=1}^B \exp(\langle \hat{e}_b^Q, \hat{e}_j^A \rangle / \tau_{QA})}, \quad (7)$$

with $\tau_{QA} = 0.07$. Questions and answers about the same topic must occupy nearby points.

Autocorrelation Alignment. We align second-order structure via Gram matrices:

$$C_Q = Z_Q^\top Z_Q / (|Q| + \epsilon), \quad C_A = Z_A^\top Z_A / (|A| + \epsilon), \quad (8)$$

where Z_Q stacks unit-normalised token embeddings of the question. The loss is

$$\mathcal{L}_{Gram} = \frac{1}{B} \sum_b \|C_Q^{(b)} - C_A^{(b)}\|_F^2. \quad (9)$$

This enforces that relational geometry of tokens inside the question matches that of its answer, pushing the model to capture latent roles (subject, object, modality) rather than surface co-occurrence only.

Code Regularisers. Usage and independence penalties from Section 2 are reused to keep codes expressive when conditioned on real language contexts.

3.3 Optimisation Behaviour

Gradients from $\mathcal{L}_{QA}$ propagate through the codebooks: if two different answers share a topic (“photosynthesis” or Chinese equivalent “guang he zuo yong”), they must align to similar pooled embeddings, pulling constituent tokens share closer code vectors. Because everything is functions of glyphs, the optimisation is language-agnostic; the network only sees strokes and shapes.

Why This Learns Meaning.

- InfoNCE collapses question–answer pairs into the same neighbourhood, so the code representation must encode semantic compatibility.
- The Gram term penalises mismatched relational geometry. For instance, if a question emphasises an agent acting on a patient, the embeddings for verbs and nouns must sit in comparable orientations to those in the answer.
- Usage and independence prevent trivial solutions (everyone to one index), forcing the network to reuse latent factors across many QA items. Shared factors coalesce into interpretable axes (e.g., animacy, polarity, aspect).

4 Diffusion over Sentence Frames

Tokens are assembled into $H \times W$ grids; the glyph CNN produces features $F \in \mathbb{R}^{B \times d \times H \times W}$.

A Bernoulli mask M with per-sample ratio $\beta \sim \mathcal{U}(\beta_{\min}, \beta_{\max})$ selects which positions to corrupt. The input to UNet is $[\tilde{F} \| M]$ where $\tilde{F} = F \odot (1 - M)$.

The UNet predicts *abla*-like residuals, trained with masked MSE:

$$\mathcal{L}_{\text{diff}} = \frac{1}{|\{i : M_i = 1\}|} \sum_{i: M_i=1} \|\hat{F}_i - F_i\|_2^2. \quad (10)$$

This encourages the network to exploit context when reconstructing masked features. As colour codes carry semantics, the diffusion head learns to fill in tokens coherently.

5 Extension to Historical or Unsegmented Scripts

5.1 Glyph Layer is the Key

All stages operate on glyph images. To ingest a new script (Oracle Bone, Khitan, Tangut), supply a glyph renderer (font or vector) and populate the glyph DB with characters encountered in the corpus. No tokenisation is required; each character is a visual symbol.

5.2 Learning Without Dictionaries

Given only monolingual text:

1. Build glyph dataset: run `scripts/glyphdb_add_tokens.py` on unique characters.
2. Train product codes on glyph reconstruction (Section 2). Shared radicals or strokes will cluster automatically.
3. Use pseudo-QA generated from the corpus: e.g., treat adjacent sentences as (context, response) pairs, or use masked spans as question/answer. The InfoNCE loss still pushes semantically related glyph groups together.
4. Once codes stabilise, train diffusion for denoising; the model learns internal language structure just from visual co-occurrence.

Without dictionaries, semantics emerge from distributional structure: questions and answers formed by context windows, sentence pairs, or self-supervised tasks provide contrastive signals.

5.3 Practical Considerations

- **Font Availability:** For ancient scripts, use raster renderings (scans) embedded as glyphs. The pipeline is agnostic to font source.
- **Corpus Size:** Contrastive learning benefits from millions of QA-like pairs. For scarce data, augment with back-translation or pseudo-instruction templates.
- **Evaluation:** Use nearest-neighbour clustering on product codes to discover radicals or semantic fields; compare to ground truth when available, or rely on human experts.

6 Derivation Summary

We summarise the total objective being minimised across stages:

$$\mathcal{L} = \mathcal{L}_{\text{codes}} + \mathcal{L}_{\text{QA}} + \lambda_{\text{Gram}}\mathcal{L}_{\text{Gram}} + \lambda_{\text{usage}}\mathcal{L}_{\text{usage}} + \lambda_{\text{indep}}\mathcal{L}_{\text{indep}} + \mathcal{L}_{\text{diff}}. \quad (11)$$

Each term propagates gradients to the shared codebooks $\{E_c\}$, making them the semantic memory. Because the glyph CNN is frozen during diffusion and fine-tuned during QA, tokens remain anchored to their visual form while gradually aligned semantically.

7 Why the Pipeline Works

1. **Visual Anchoring:** Tokens remain tied to glyphs, so the representation respects orthographic similarity (stroke sharing).
2. **Contrastive Pull:** InfoNCE objectives across glyph-code and QA pairs enforce that meaning-bearing contexts cluster.
3. **Structural Regularisation:** Gram alignment and independence penalties prevent degenerate minima, promoting interpretable subspace factors.
4. **Diffusion Closure:** The denoiser validates that the learned codes retain enough information to reconstruct sentences from corrupted fragments. This demonstrates that semantics are not destroyed by discretisation.

8 Future Directions

- **Orthography Discovery:** Run clustering on colour codes to propose radical decompositions for undocumented scripts.
- **Interactive Translation:** Use diffusion infilling conditioned on bilingual seeds to bootstrap lexicons for under-studied languages.
- **Probabilistic Grammars:** Fit simple probabilistic models (e.g., HMMs) over the colour-code sequences to extract latent grammatical categories.

9 Conclusion

The ILM codebase already optimises a principled objective that aligns visual token embeddings with semantic structure purely from images and QA supervision. Because the glyph interface is script-agnostic, the same method can ingest Oracle Bone inscriptions, Tangut texts, or any logographic script without dictionaries: any corpora of glyph sequences can serve as self-supervision for the colour-code memory table, providing an automatic path toward meaning discovery.

References

- [1] van den Oord et al. *Representation Learning with Contrastive Predictive Coding*.
- [2] Jang et al. *Categorical Reparameterization with Gumbel-Softmax*.
- [3] Liu et al. *Glyph-aware Multilingual Models*.